

Sequence to Sequence Learning with Neural Networks

用神经网络进行序列到序列学习

Ilya Sutskever Oriol Vinyals Quoc V. Le

Google

{ilyasu, vinyals, qvl}@google.com

伊利亚·苏茨克维尔·奥里奥尔·维尼亚尔斯·夸克·V·黎(均来自 Google)

Advances in Neural Information Processing Systems 27 (NIPS 2014), pp. 3104–3112

Abstract

摘要

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.

深度神经网络 (DNN) 是非常强大的模型,在许多困难的机器学习任务上都取得了卓越的表现。虽然只要有足够大的有标注训练集,DNN 通常就能工作得很好,但它无法被直接用来完成“序列到序列”的映射。本文提出了一种对序列结构几乎不做假设的、端到端的通用序列学习方法。我们的方法使用一个多层长短期记忆网络 (LSTM) 将输入序列映射成一个固定维度的向量,然后用另一个深度 LSTM 从该向量“解码”出目标序列。主要实验结果为:在 WMT'14 英语—法语翻译任务上,由 LSTM 生成的译文在整个测试集上达到了 34.8 的 BLEU 分数,其中由于词表外的词对分数造成了一定的惩罚。值得一提的是,LSTM 在长句上并未遇到困难。作为对比,基于短语的传统统计机器翻译 (SMT) 系统在同一数据集上的 BLEU 分数为 33.3。当我们使用 LSTM 对上述 SMT 系统所产生的 1000 条候选翻译进行重排序 (re-rank) 时,其 BLEU 分数可以提升到 36.5,非常接近当时该任务的最佳公开结果。LSTM 还学到了对词序敏感、对主动/被动语态相对稳健的短语级与句子级表示。最后,我们发现:在所有源语句(但不改变目标语句)中反转词序可以显著提升 LSTM 的性能,这是因为反转词序在源语言和目标语言之间引入了许多“短期依赖”,从而大大降低了优化问题的难度。

1. Introduction

引言

Deep Neural Networks (DNNs) are extremely powerful machine learning models that achieve excellent performance on difficult problems such as speech recognition and visual object recognition. DNNs are powerful because they can perform arbitrary parallel computation for a modest number of steps. A surprising example of the power of DNNs is their ability to sort N N -bit numbers using only 2 hidden layers of quadratic size. So, while neural networks are related to conventional statistical models, they learn an intricate computation. Furthermore, large DNNs can be trained with supervised backpropagation whenever the labeled training set has enough information to specify the network's parameters. Thus, if there exists a parameter setting of a large DNN that achieves good results (for example, because humans can solve the task very rapidly), supervised backpropagation will find these parameters and solve the problem.

The main result of this work is the following. On the WMT'14 English to French translation task, we obtained a BLEU score of 34.81 by directly extracting translations from an ensemble of 5 deep LSTMs (with 384M parameters and 8,000 dimensional state each) using a simple left-to-right beam-search decoder. This is by far the best result achieved by direct translation with large neural networks. For comparison, the BLEU score of an SMT baseline on this dataset is 33.30. The 34.81 BLEU score was achieved by an LSTM with a vocabulary of 80k words, so the score was penalized whenever the reference translation contained a word not covered by these 80k. This result shows that a relatively unoptimized small-vocabulary neural network architecture which has much room for improvement outperforms a phrase-based SMT system.

本文的主要实验结果如下:在 WMT ' 14 英语—法语翻译任务上,通过 5 个深度 LSTM(每个具有 3.84 亿参数和 8000 维隐藏状态)组成的集成模型,使用一个简单的从左到右的束搜索 (beam search) 解码器直接抽取译文,获得了 34.81 的 BLEU 分数。这是迄今为止直接使用大型神经网络进行翻译所取得的最好结果。作为对比,基于短语的统计机器翻译基线系统在同一数据集上的 BLEU 分数为 33.30。该 34.81 的 BLEU 分数是由词表为 80 000 词的 LSTM 取得的,因此每当参考译文中含有这 80 000 词之外的词时,分数就会受到惩罚。这一结果表明,一个相对未经精细调优、词表较小、但仍有很大改进空间的神经网络架构,能够超过基于短语的统计机器翻译系统。

Finally, we used the LSTM to rescore the publicly available 1000-best lists of the SMT baseline on the same task. By doing so, we obtained a BLEU score of 36.5, which improves the baseline by 3.2 BLEU points and is close to the previous best published result on this task (which is 37.0).

最后,我们使用 LSTM 对同一任务上 SMT 基线系统公开的 1000-best 候选列表进行重排序。由此得到的 BLEU 分数为 36.5,相对基线提升了 3.2 个 BLEU 点,非常接近该任务当时最佳的公开结果(37.0)。

Surprisingly, the LSTM did not suffer on very long sentences, despite the recent experience of other researchers with related architectures. We were able to do well on long sentences because we reversed the order of words in the source sentence but not the target sentences in the training and test set. By doing so, we introduced many short term dependencies that made the optimization problem much simpler (see sec. 2 and sec. 3.3). As a result, SGD could learn LSTMs that had no trouble with long sentences. The simple trick of reversing the words in the source sentence is one of the key technical contributions of this work.

令人惊讶的是,LSTM 在很长的句子上并没有出现性能下降,而其他研究者近期在类似架构上的经验并非如此。我们之所以能在长句上表现良好,是因为我们在训练集和测试集中反转了源语句中的词序,但不反转目标句的词序。这样做在数据中引入了大量短期依赖,显著降低了优化问题的复杂度(详见第 2 节和第 3.3 节)。其结果是,SGD 可以成功训练出能从容应对长句的 LSTM。在源语句中反转词序这一看似简单的小技巧,是本文的关键技术贡献之一。

A useful property of the LSTM is that it learns to map an input sentence of variable length into a fixed-dimensional vector representation. Given that translations tend to be paraphrases of the source sentences, the translation objective encourages the LSTM to find sentence representations that capture their meaning, as sentences with similar meanings are close to each other while different sentences meanings will be far. A qualitative evaluation supports this claim, showing that our model is aware of word order and is fairly invariant to the active and passive voice.

LSTM 的一个有用性质是:它能学会将变长的输入句映射为一个固定维度的向量表示。由于翻译往往是对源句的转述,翻译目标会促使 LSTM 找到能捕捉句意的句子表示——含义相近的句子彼此距离较近,而含义不同的句子彼此距离较远。定性实验支持了这一说法,显示我们的模型能感知词序,并且对主动 / 被动语态的替换相对稳健。

2. The Model

模型

The Recurrent Neural Network (RNN) is a natural generalization of feedforward neural networks to sequences. Given a sequence of inputs ($x_1, \dots, x_T$), a standard RNN computes a sequence of outputs ($y_1, \dots, y_T$) by iterating the following equation: $h_t = \text{sigm}(W^{hx} x_t + W^{hh} h_{t-1})$; $y_t = W^{yh} h_t$. The RNN can easily map sequences to sequences whenever the alignment between the inputs and the outputs is known ahead of time. However, it is not clear how to apply an RNN to problems whose input and the output sequences have different lengths with complicated and non-monotonic relationships.

循环神经网络 (RNN) 是前馈神经网络向序列的自然推广。给定一个输入序列 $(x_1, \dots, x_T)$, 标准 RNN 通过反复迭代以下方程来计算输出序列 $(y_1, \dots, y_T)$: $h_t = \text{sigm}(W^{hx} x_t + W^{hh} h_{t-1})$; $y_t = W^{yh} h_t$ 。只要输入与输出之间的对齐关系事先已知, RNN 就能容易地完成序列到序列的映射。然而, 对于输入序列和输出序列长度不同、且关系复杂且非单调的问题, RNN 应当如何使用并不明朗。

The simplest strategy for general sequence learning is to map the input sequence to a fixed-sized vector using one RNN, and then to map the vector to the target sequence with another RNN (this approach has also been taken by Cho et al. (2014)). While it could work in principle since the RNN is provided with all the relevant information, it would be difficult to train the RNNs due to the resulting long term dependencies (fig. 1). However, the Long Short-Term Memory (LSTM) is known to learn problems with long range temporal dependencies, so an LSTM may succeed in this setting.

对于一般性的序列学习, 最简单的策略是: 用一个 RNN 将输入序列映射为一个固定大小的向量, 再用另一个 RNN 将该向量映射为目标序列 (Cho 等人 (2014) 也采用了这一方案)。原则上这种做法应该可行, 因为 RNN 在每一步都能获得所有相关信息; 然而, 由于输入与输出之间存在长时间跨度的依赖关系 (如图 1 所示), 这种模型的训练会非常困难。但长短期记忆网络 (LSTM)

已被证明能够在具有长程时间依赖性的问题上成功学习, 因此在该场景下 LSTM 应当能够成功。

The goal of the LSTM is to estimate the conditional probability $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ where $(x_1, \dots, x_T)$ is an input sequence and $y_1, \dots, y_{T'}$ is its corresponding output sequence whose length T' may differ from T . The LSTM computes this conditional probability by first obtaining the fixed-dimensional representation v of the input sequence $(x_1, \dots, x_T)$ given by the last hidden state of the LSTM, and then computing the probability of $y_1, \dots, y_{T'}$ with a standard LSTM-LM formulation whose initial hidden state is set to the representation v of $x_1, \dots, x_T$: $p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1})$.

LSTM 的目标是估计条件概率 $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$, 其中 $(x_1, \dots, x_T)$ 是输入序列, $y_1, \dots, y_{T'}$ 是对应的输出序列, 其长度 T' 可能与 T 不相等。LSTM 通过以下方式计算这个条件概率: 首先, 输入序列 $(x_1, \dots, x_T)$ 由 LSTM 的最后一个隐藏状态给出其固定维度的向量表示 v ; 然后, 以 v 作为初始隐藏状态, 使用标准的 LSTM 语言模型公式来计算 $y_1, \dots, y_{T'}$ 的概率: $p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1})$ 。

In this equation, each $p(y_t | v, y_1, \dots, y_{t-1})$ distribution is represented with a softmax over all the words in the vocabulary. We use the LSTM formulation from Graves (2013). Note that we require that each sentence ends with a special end-of-sentence symbol “”, which enables the model to define a distribution over sequences of all possible lengths. The overall scheme is outlined in fig. 1, where the shown LSTM computes the representation of “A”, “B”, “C”, “” and then uses this representation to compute the probability of “W”, “X”, “Y”, “Z”, “”.

在上述公式中, 每个分布 $p(y_t | v, y_1, \dots, y_{t-1})$ 用词表上所有词上的 softmax 来表示。我们采用 Graves (2013) 所提出的 LSTM 公式。需要注意的是, 我们要求每个句子都以一个特殊的句末符号 “” 结尾, 这样模型就可以对所有可能长度的序列定义分布。整个方案的示意图如图 1 所示: 图中 LSTM 先计算出 “A”、“B”、“C”、“” 的表示, 然后再用该表示计算 “W”、“X”、“Y”、“Z”、“” 的概率。

Our actual models differ from the above description in three important ways. First, we used two different LSTMs: one for the input sequence and another for the output sequence, because doing so increases the number of model parameters at negligible computational cost and makes it natural to train the LSTM on multiple language pairs simultaneously. Second, we found that deep LSTMs significantly outperformed shallow LSTMs, so we chose an LSTM with four layers. Third, we found it extremely valuable to reverse the order of the words of the input sentence. So for example, instead of mapping the sentence a, b, c to the sentence α, β, γ , the LSTM is asked to map c, b, a to α, β, γ , where α, β, γ is the translation of a, b, c. This way, a is in close proximity to α , b is fairly close to β , and so on, a fact that makes it easy for SGD to “establish communication” between the input and the output. We found this simple data transformation to greatly improve the performance of the LSTM.

我们实际的模型与上述描述在三个重要方面有所不同。第一, 我们使用了两个不同的 LSTM:

一个用于输入序列, 另一个用于输出序列。这样做在几乎不增加计算开销的前提下增加

了模型参数量, 并且使同时在多种语言对上训练 LSTM 变得自然。第二, 我们发现深度 LSTM 显著优于浅层 LSTM, 因此选用了 4 层的 LSTM。第三, 我们发现将源句的词序反转非常有价值。举例来说, 不是将句子 a, b, c 映射为 α, β, γ , 而是要求 LSTM 将 c, b, a 映射到 α, β, γ (其中 α, β, γ 是 a, b, c 的翻译)。这样, a 就与 α 在位置上靠得很近, b 也与 β 相对靠近, 以此类推。这一事实让 SGD

很容易在输入与输出之间 “建立通信”。我们发现, 这种简单的数据变换可以极大地提升 LSTM 的性能。

3. Experiments

实验

We applied our method to the WMT'14 English to French MT task in two ways. We used it to directly translate the input sentence without using a reference SMT system and we used it to rescore the n-best lists of an SMT baseline. We report the accuracy of these translation methods, present sample translations, and visualize the resulting sentence representation.

我们将本文的方法以两种方式应用于 WMT'14 英—法机器翻译任务。一是直接对输入句进行翻译,不使用任何参考 SMT 系统;二是对一个 SMT 基线系统所产生的 n-best 候选列表进行重排序。我们报告这些翻译方法的准确度,给出若干翻译样例,并对所学到的句子表示进行可视化。

3.1 Dataset Details

数据集细节

We used the WMT'14 English to French dataset. We trained our models on a subset of 12M sentences consisting of 348M French words and 304M English words, which is a clean “selected” subset from Schwenk (2014). We chose this translation task and this specific training set subset because of the public availability of a tokenized training and test set together with 1000-best lists from the baseline SMT system.

我们使用 WMT'14 英—法数据集。我们在由 1200 万句组成的子集(包含 3.48 亿法语词和 3.04 亿英语词)上训练模型,这是 Schwenk (2014) 提供的一个干净的“精选”子集。选择这个翻译任务和这个特定训练子集的原因是:它公开提供了分词后的训练集与测试集,并且能够获得 SMT 基线系统的 1000-best 候选列表。

As typical neural language models rely on a vector representation for each word, we used a fixed vocabulary for both languages. We used 160,000 of the most frequent words for the source language and 80,000 of the most frequent words for the target language. Every out-of-vocabulary word was replaced with a special “UNK” token.

由于典型的神经语言模型依赖每个词的向量表示,我们对两种语言都使用了固定词表。源语言使用 160 000 个最高频词,目标语言使用 80 000 个最高频词。每个词表外的词都被替换为特殊标记“UNK”。

3.2 Decoding and Rescoring

解码与重排序

The core of our experiments involved training a large deep LSTM on many sentence pairs. We trained it by maximizing the log probability of a correct translation T given the source sentence S , so the training objective is $(1/|S|) \sum_{(T,S) \in S} \log p(T|S)$, where S is the training set. Once training is complete, we produce translations by finding the most likely translation according to the LSTM: $T^* = \operatorname{argmax}_T p(T|S)$. We search for the most likely translation using a simple left-to-right beam search decoder which maintains a small number B of partial hypotheses, where a partial hypothesis is a prefix of some translation. At each timestep we extend each partial hypothesis in the beam with every possible word in the vocabulary. This greatly increases the number of the hypotheses so we discard all but the B most likely hypotheses according to the model's log probability. As soon as the “” symbol is appended to a hypothesis, it is removed from the beam and is added to the set of complete hypotheses. While this decoder is approximate, it is simple to implement. Interestingly, our system performs well even with a beam size of 1, and a beam of size 2 provides most of the benefits of beam search.

我们实验的核心是在大量句对上训练一个大型的深度 LSTM。我们通过对数似然最大化进行训练:给定源句 S , 最大化正确译文 T 的对数概率, 目标函数为 $(1/|S|) \sum_{(T,S) \in S} \log p(T|S)$, 其中 S 是训练集。训练完成后, 通过寻找 LSTM 下的最可能译文来生成译文: $T^* = \operatorname{argmax}_T p(T|S)$ 。我们用一个简单的从左到右的束搜索解码器来寻找最可能的译文, 该解码器维护少量 B 个“部分假设”, 其中部分假设是某个译文的前缀。在每个时间步, 我们将束中的每个部分假设用词表中的每一个可能词进行扩展, 这会导致候选数大幅增加, 于是我们按模型对数概率只保留 B 个最可能的假设。一旦某个假设后接了 “”, 就将它从束中移出, 并加入到完整假设集合中。虽然这个解码器是近似的, 但实现起来非常简单。有趣的是, 即使束大小为 1, 我们的系统也能表现良好; 而束大小为 2 就能获得束搜索的绝大部分收益。

We also used the LSTM to rescore the 1000-best lists produced by the baseline system. To rescore an n-best list, we computed the log probability of every hypothesis with our LSTM and took an even average with their score and the LSTM's score.

我们还使用 LSTM 对基线系统产生的 1000-best 候选列表进行重排序。对 n-best 列表进行重排序时, 我们用 LSTM 计算每一条假设的对数概率, 并将其分数与 LSTM 分数取算术平均。

3.3 Reversing the Source Sentences

反转源句词序

While the LSTM is capable of solving problems with long term dependencies, we discovered that the LSTM learns much better when the source sentences are reversed (the target sentences are not reversed). By doing so, the LSTM's test perplexity dropped from 5.8 to 4.7, and the test BLEU scores of its decoded translations increased from 25.9 to 30.6.

尽管 LSTM 具备解决长程依赖问题的能力,我们发现,当源语句词序被反转时(目标语句词序 保持不变),LSTM 学得更好。这样处理后,LSTM 的测试困惑度从 5.8 下降到 4.7,其解码 所得译文的测试 BLEU 分数从 25.9 提升到 30.6。

While we do not have a complete explanation to this phenomenon, we believe that it is caused by the introduction of many short term dependencies to the dataset. Normally, when we concatenate a source sentence with a target sentence, each word in the source sentence is far from its corresponding word in the target sentence. As a result, the problem has a large “minimal time lag”. By reversing the words in the source sentence, the average distance between corresponding words in the source and target language is unchanged. However, the first few words in the source language are now very close to the first few words in the target language, so the problem's minimal time lag is greatly reduced. Thus, backpropagation has an easier time “establishing communication” between the source sentence and the target sentence, which in turn results in substantially improved overall performance.

虽然我们尚无完整的解释,但我们相信这一现象是因为在数据集中引入了大量“短期依赖”。通常情况下,当我们把源句和目标句拼接起来时,源句中的每个词都离它在目标句中对应的词很远,因此问题具有较大的“最小时间延迟”。反转源句中的词序后,源语言与目标语言中对应词之间的平均距离并未改变。然而,源语言开头的若干词现在与目标语言开头的若干词非常接近,于是问题的“最小时间延迟”被显著缩短。这样,反向传播就更容易在源句与目标句之间“建立通信”,从而显著提升整体性能。

Initially, we believed that reversing the input sentences would only lead to more confident predictions in the early parts of the target sentence and to less confident predictions in the later parts. However, LSTMs trained on reversed source sentences did much better on long sentences than LSTMs trained on the raw source sentences (see sec. 3.7), which suggests that reversing the input sentences results in LSTMs with better memory utilization.

最初我们以为,反转输入句只会让目标句前部分的预测更自信、目标句后部分的预测更不自信。然而,在反转源句上训练的 LSTM 在长句上反而比在原始源句上训练的 LSTM 表现更好(见第 3.7 节)。这表明,反转源句能够获得对内存利用更高效的 LSTM。

3.4 Training Details

训练细节

We found that the LSTM models are fairly easy to train. We used deep LSTMs with 4 layers, with 1000 cells at each layer and 1000 dimensional word embeddings, with an input vocabulary of 160,000 and an output vocabulary of 80,000. Thus the deep LSTM uses 8000 real numbers to represent a sentence. We found deep LSTMs to significantly outperform shallow LSTMs, where each additional layer reduced perplexity by nearly 10%, possibly due to their much larger hidden state. We used a naive softmax over 80,000 words at each output. The resulting LSTM has 384M parameters of which 64M are pure recurrent connections (32M for the “encoder” LSTM and 32M for the “decoder” LSTM). The complete training details are given below:

我们发现 LSTM 模型相当容易训练。我们使用 4 层的深度 LSTM,每层 1000 个单元,词嵌入维度为 1000;源端词表为 160 000,目标端词表为 80 000。因此,深度 LSTM 用 8000 个实数表示一个句子。我们发现深度 LSTM 显著优于浅层 LSTM,每增加一层,困惑度几乎下降 10%,这可能得益于它们更大的隐藏状态。在每个输出位置我们使用朴素的、词表大小为 80 000 的 softmax。最终得到的 LSTM 共有 3.84 亿参数,其中 6400 万为纯粹的循环连接(“编码器” LSTM 占 3200 万,“解码器” LSTM 占 3200 万)。完整训练细节如下:

- We initialized all of the LSTM's parameters with the uniform distribution between -0.08 and 0.08 .
- We used stochastic gradient descent without momentum, with a fixed learning rate of 0.7 . After 5 epochs, we begun halving the learning rate every half epoch. We trained our models for a total of 7.5 epochs.
- We used batches of 128 sequences for the gradient and divided it the size of the batch (namely, 128).
- Although LSTMs tend to not suffer from the vanishing gradient problem, they can have exploding gradients. Thus we enforced a hard constraint on the norm of the gradient by scaling it when its norm exceeded a threshold. For each training batch, we compute $s = \frac{\|g\|}{128}$, where g is the gradient divided by 128. If $s >$

5, we set $g = 5g/s$. • Different sentences have different lengths. Most sentences are short (e.g., length 20–30) but some sentences are long (e.g., length > 100), so a minibatch of 128 randomly chosen training sentences will have many short sentences and few long sentences, and as a result, much of the computation in the minibatch is wasted. To address this problem, we made sure that all sentences in a minibatch are roughly of the same length, yielding a 2× speedup.

- 我们用 $(-0.08, 0.08)$ 区间上的均匀分布初始化 LSTM 的所有参数。

我们使用不带动量的随机梯度下降(SGD),固定学习率为 0.7。训练 5 个 epoch 后,我们开始每半个 epoch 将学习率减半;模型一共训练 7.5 个 epoch。

- 梯度按大小为 128 的序列批量计算,并除以批大小(即 128)。
- 虽然 LSTM 通常不存在梯度消失的问题,但可能会出现梯度爆炸。因此我们对梯度的范数施加了一个硬约束:当梯度范数超过阈值时对其做缩放。具体做法是:对每个训练批,计算 $s = \|g\|_2$, 其中 g 为除以 128 后的梯度。若 $s > 5$, 则令 $g = 5g/s$ 。
- 不同句子长度差别很大。大部分句子较短(例如 20–30 个词),但有些句子很长(例如长度 > 100)。因此,从训练集中随机抽取 128 句构成的 minibatch 中短句居多、长句稀少,造成 minibatch 中大量计算被浪费。为解决这一问题,我们使每个 minibatch 内的句子长度大致接近,从而获得约 2 倍的加速。

3.5 Parallelization

并行化

A C++ implementation of deep LSTM with the configuration from the previous section on a single GPU processes a speed of approximately 1,700 words per second. This was too slow for our purposes, so we parallelized our model using an 8-GPU machine. Each layer of the LSTM was executed on a different GPU and communicated its activations to the next GPU / layer as soon as they were computed. Our models have 4 layers of LSTMs, each of which resides on a separate GPU. The remaining 4 GPUs were used to parallelize the softmax, so each GPU was responsible for multiplying by a 1000×20000 matrix. The resulting implementation achieved a speed of 6,300 (both English and French) words per second with a minibatch size of 128. Training took about ten days with this implementation.

对于上一节配置的深度 LSTM,基于 C++ 的实现在单 GPU 上每秒大约能处理 1 700 个词,这对我们来说太慢了。因此,我们将模型并行化到一台 8 卡 GPU 的机器上。LSTM 的每一层运行在不同的 GPU 上,一旦计算出激活值,立即将其发送到对应下一层 / 下一 GPU。我们的模型有 4 层 LSTM,每层单独占用一块 GPU;其余 4 块 GPU 用来并行 softmax,即每块 GPU 负责与一个 $1000 \times 20\,000$ 维矩阵相乘。最终实现的处理速度为 6 300 个(英 / 法)词 / 秒,minibatch 大小为 128。该实现训练大约需要 10 天。

3.6 Experimental Results

实验结果

We used the cased BLEU score to evaluate the quality of our translations. We computed our BLEU scores using multi-bleu.pl on the tokenized predictions and ground truth. This way of evaluating the BLEU score is consistent with Cho et al. (2014) and Bahdanau et al. (2014), and reproduces the 33.3 score of Schwenk (2014). However, if we evaluate the best WMT'14 system (whose predictions can be downloaded from statmt.org) in this manner, we get 37.0, which is greater than the 35.8 reported by statmt.org.

我们使用区分大小写的 BLEU 分数来评估译文质量。使用 multi-bleu.pl 在分词后的预测结果与参考译文上计算 BLEU 分数。这种评估方式与 Cho 等人(2014)和 Bahdanau 等人 (2014)一致,并复现了 Schwenk (2014) 的 33.3 这一分数。然而,如果用同样方式评估 WMT ' 14 最好的系统(其预测可以从 statmt.org 下载),我们会得到 37.0,高于 statmt.org 报告的 35.8。

The results are presented in Table 1 and Table 2. Our best results are obtained with an ensemble of LSTMs that differ in their random initializations and in the random order of minibatches. While the decoded translations of the LSTM ensemble do not outperform the best WMT'14 system, it is the first time that a pure neural translation system outperforms a phrase-based SMT baseline on a large scale MT task by a sizeable margin, despite its inability to handle out-of-vocabulary words. The LSTM is within 0.5 BLEU points of the best WMT'14 result if it is used to rescore the 1000-best list of the baseline system.

实验结果如表 1 和表 2 所示。我们最好的结果来自一个 LSTM 集成模型,其中的 LSTM 在随机初始化和 minibatch 随机顺序上互不相同。虽然 LSTM 集成模型的解码译文仍未超过 WMT ' 14 最好的系统,但它首次在如此大规模的机器翻译任务上,以明显的优势超过了一个基于短语的 SMT 基线系统——尽管它还无法处理词表外的词。当 LSTM 被用于对基线系统的 1000-best 候选列表进行重排序时,其在 BLEU 上仅与 WMT ' 14 最佳结果相差 0.5 点。

结论

在本文中,我们证明:一个大型的、词表有限的深度 LSTM,几乎不依赖任何问题结构上的假设,就能在一个大规模的机器翻译任务上超过一个词汇不受限制的标准 SMT 系统。我们这种简单的基于 LSTM 的方法在机器翻译上的成功表明:只要拥有足够的训练数据,它也应该能在许多其他序列学习问题上表现良好。

反转源语句词序所带来的提升幅度令我们感到惊讶。由此我们得出结论:寻找一种具有尽可能多短期依赖的问题编码方式非常重要,因为短期依赖会使学习问题变得简单得多。具体而言,虽然我们无法在未反转的翻译问题上成功训练一个标准 RNN(如图 1 所示),但我们认为:只要将源语句反转,标准 RNN 也应当能够被轻易地训练起来(尽管我们没有进行实验验证)。

LSTM 仍能正确翻译很长的句子,这一点也令我们感到惊讶。我们原本以为 LSTM 由于其有限的记忆容量,会在长句上失败;其他研究者在与我们类似的模型上也报告过在长句上表现不佳。然而,在反转源句的数据集上训练的 LSTM 翻译长句却毫无困难。

最重要的是,我们证明了一个简单、直接、且相对未经精细调优的方法可以超过 SMT 系统,因此后续的工作有望带来更高的翻译准确度。这些结果表明,本文的方法很可能在其他具有挑战性的序列到序列问题上同样表现良好。

致谢

我们感谢 Samy Bengio、Jeff Dean、Matthieu Devin、Geoffrey Hinton、Nal Kalchbrenner、Thang Luong、Wolfgang Macherey、Rajat Monga、Vincent Vanhoucke、Peng Xu、Wojciech Zaremba 以及 Google Brain 团队所提供的宝贵意见与讨论。